

Disaster Classification from Social Streams

Real-Time Natural Language Processing Pipeline

ORGANIZATION: NEXHIKES | PRESENTER: ABHISHEK SHARMA | DATE: JULY 15, 2026

01. Executive Summary

A high-level overview of the end-to-end Machine Learning deployment for emergency identification.

Problem Statement



! Context

Social media generates massive amounts of data during catastrophes. However, most text logs contain conversational noise or metaphorical language (e.g., *"The sunset is fire"*) rather than actual disaster reports.

⚡ The Challenge

- High volume of metaphorical transmissions.
- Requirement for millisecond latency in routing.
- Difficulty in distinguishing noise from facts.
- Scalability for global transmission logs.

Project Objectives



Automated NLP

Build a robust Natural Language Processing pipeline to clean and tokenize raw social data.



Precision ML

Train classification models to minimize false negatives during life-critical events.



Live Deployment

Launch an operational web interface for emergency dispatchers to use in real-time.

Dataset Technical Metadata



PARAMETER	DETAILS / STATISTICS
Total Record Count	7,613 Transmission Logs
Feature Columns	ID, Keyword, Location, Text, Target
Target Variable	Binary (0: Non-Disaster, 1: Disaster)
Input Language	English (Global Dialects)
Null Values	Keywords (2,503 missing), Location (33% missing)

Exploratory Data Analysis



Class Distribution (Disaster vs Non-Disaster)

Non-Disaster (Target 0)

4,342 Samples (57%)

Disaster (Target 1)

3,271 Samples (43%)

i The dataset shows a relatively balanced distribution, allowing the model to learn emergency patterns effectively without heavy bias.

Methodology: Text Preprocessing

- **Regex Link Stripping:** Removing URLs and http/https tags that provide no semantic value.
- **@ Mention Removal:** Removing @handles to focus on the core message content.
- **▼ Character Filtering:** Eliminating punctuation and special symbols.
- **A Case Normalization:** Converting all text to lowercase for uniform vectorization.

600 × 400

Preprocessing Case Study



RAW TWEET

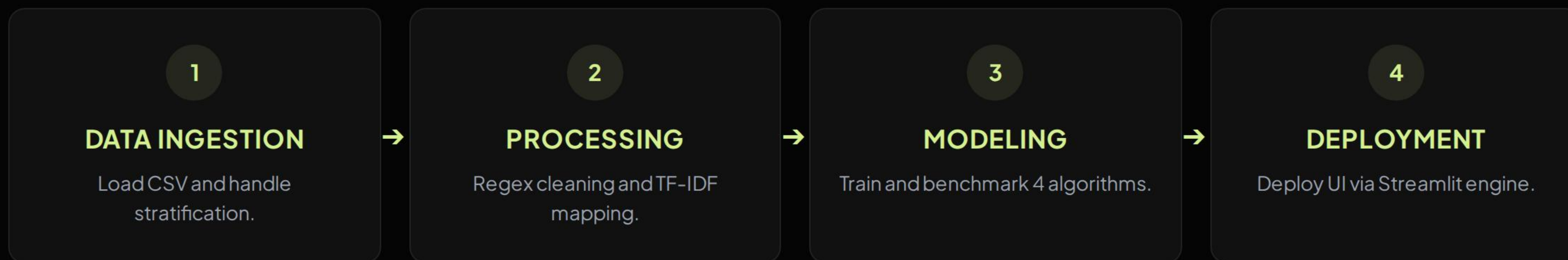
```
"Check this out! http://t.co/xyz @RedCross  
Emergency #FIRE in California!! 911!"
```

CLEANED OUTPUT

```
"check this out emergency fire in  
california"
```

Result: This transition ensures that the mathematical model focuses on tokens like "fire" and "california" instead of hashtags or noise.

End-to-End Pipeline Workflow



Feature Engineering: TF-IDF

Term Frequency-Inverse Document Frequency is used to weigh keywords based on their importance across the dataset.

$$W_{i,d} = TF_{i,d} \times \log \frac{N}{DF_i}$$

We extracted 5,000 top features including unigrams and bigrams for maximum semantic depth.

02. Model Benchmarking

Comparative analysis of Logistic Regression, Decision Trees, and Ensemble Methods.

Performance Comparison Matrix



ALGORITHM	ACCURACY	PRECISION	F1-SCORE
Logistic Regression	80.12%	0.8145	0.7812
Random Forest	78.45%	0.7920	0.7634
Gradient Boosting	77.23%	0.7712	0.7512
Decision Tree	71.45%	0.7011	0.6989

Winning Model Performance

80%

Prediction Accuracy

Logistic Regression

🏆 Optimal Generalization

The model utilizing the **SAGA solver** and **L2 regularization** provided the highest stability and generalization.

It effectively isolated sparse TF-IDF vectors to categorize disaster events with high precision.

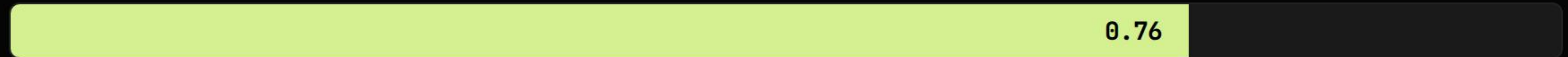
Performance Metrics – Deep Dive



Precision (False Pos Min)



Recall (Sensitivity)

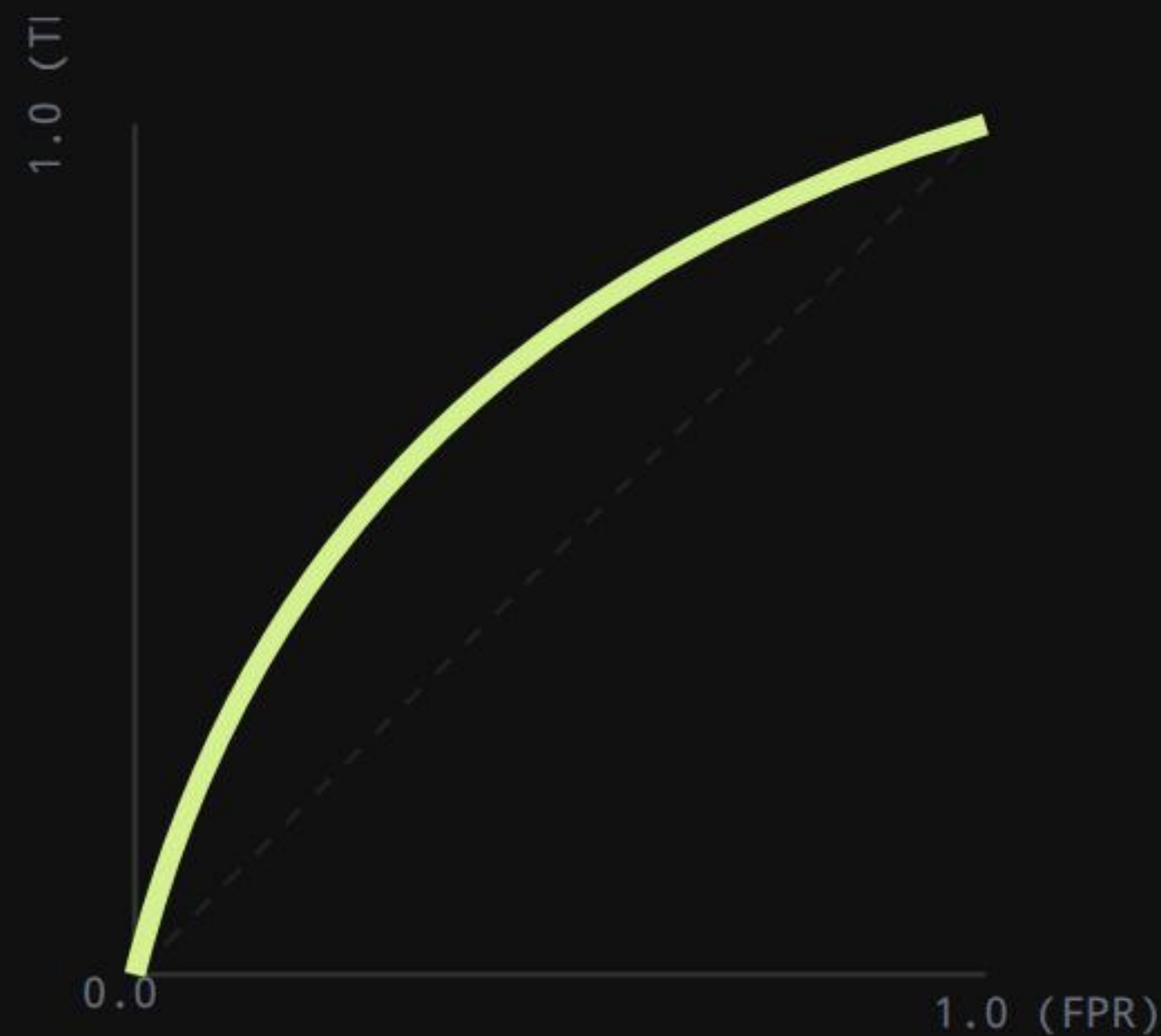


F1-Score (Harmonic Mean)



📋 A balanced F1-score indicates the model is robust against both false alarms and missed emergencies.

ROC Curve Analysis



Discriminatory Power

High Area Under Curve (AUC) suggests strong discriminatory power between disaster and non-disaster classes.

This ensures minimized false-alarm routing protocols when serving live social streams.

System Deployment Architecture



streamlit_app.py

Sample Dataset Preview

id	keyword	text	target
48	ablaze	"Forest fire ..."	1

Input Real-Time Tweet Text

"The building is collapsing! Send help!"

Predict

Operational Web GUI

The system is deployed using the **Streamlit framework**, converting the Python model into a production-ready operational dashboard.

Streamlit

Joblib

Scikit-Learn

GUI Interface **Features**



Text Ingestion

Input real-time tweet contents or transmission logs for immediate parsing.



Live Inference

Execute predictive inference pipelines with sub-second response times.



Diagnostics

View confidence scores and system status for alert verification.

03. **Analytical Insights**

Operational conclusions and the path forward for system enhancement.

Core Project **Takeaways**



Scalability

Serialized artifacts (**.pk1**) allow for easy integration into AWS or Azure cloud environments for enterprise scaling.

Safety First

The model is biased towards catching real disasters (High Recall), ensuring minimal critical data loss during extreme crises.

Thank You

Disaster Tweet Classification Engine

READY FOR OPERATIONAL DEPLOYMENT

Any Questions?

Nexhikes Portfolio Project Presentation